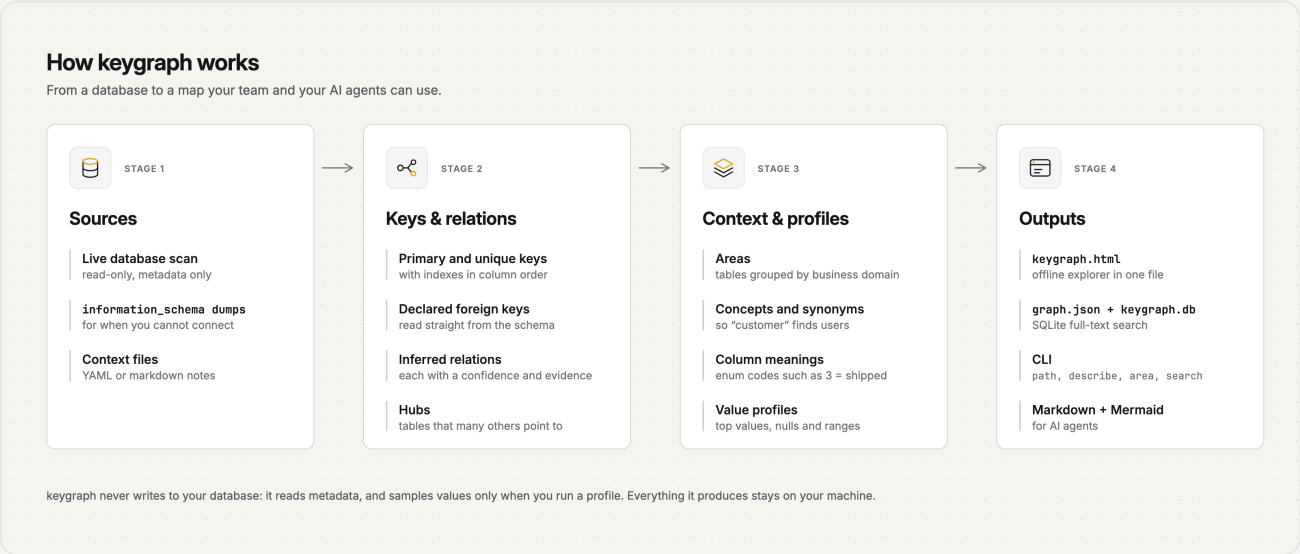
keygraph

How keygraph works

From a database to a map that people and AI agents can use. Each step shows real output from the demo webshop in examples/shop.



1. What it is
2. Connect a source
3. Keys, indexes and inferred relations
4. Hubs and areas
5. Business context and value profiles
6. Code index and query log
7. The six explorer views
8. How an agent uses it

Open source, MIT licence. Everything shown here is fictional.

OVERVIEW

keygraph turns a database's keys into a map you can query

It reads a database's structure, works out how its tables relate, adds what the business calls things, and writes an explorer for people and plain files for AI agents. It does this without calling an LLM, so the same input always gives the same output.

35	56	4	1	1
tables in 3 schemas	relations: 18 declared, 38 inferred	business areas	hub table (users)	mirror schema (reporting)

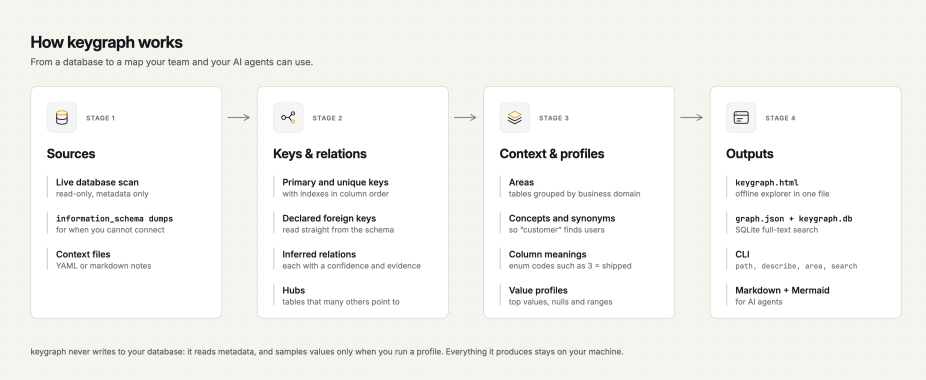
What you get

- **A map.** Every primary key, unique key and index (with column order), every declared foreign key, and the relations that were never declared, each with a confidence score and the evidence behind it.
- **Meaning.** Concepts, metrics, column codes ("3 = shipped") and what one row of a table is, so a search for "revenue" finds the right table.
- **Real values.** Top values, empty rates and ranges sampled from the data, with personal data masked.

What it never does

- It never writes to your database. It reads metadata, and it samples values only when you run `profile`, with one bounded SELECT per table.
- It never sends anything anywhere. The explorer is one HTML file that works offline.
- It never guesses silently. A relation that is only a guess is drawn dashed and carries its score.

Every example in this guide comes from a made-up webshop in `examples/shop`. You can rebuild all of it in a minute.



STEP 1

Connect a source: scan a live database or load dumps

keygraph needs the structure of your tables and nothing else. Point it at a database it can read, or hand it files that came from information_schema.

Live database (read-only)

```
keygraph scan --url postgresql://user@host/db --schema public
```

information_schema dumps, when you cannot connect

```
keygraph load --dir out \
  --columns columns.json --statistics statistics.json \
  --key-columns key_columns.json --tables tables.json \
  --context context.yaml
```

Dumps use information_schema field names. `--columns-map`, `--flatten` and `--where` adapt other shapes. Without STATISTICS, keygraph falls back to COLUMN_KEY. Without KEY_COLUMN_USAGE, every relation is inferred.

What it printed for the demo

```
$ keygraph load --dir out --columns ... --context context.yaml
keygraph: 35 tables, 224 columns, 70 indexes, 56 edges {'inferred': 38,
'declared': 18} -> examples/shop/out/ (0.4s)
```

Sources

SOURCE	HOW
MySQL, MariaDB	information_schema
PostgreSQL	information_schema, pg_catalog
SQLite, SQL Server	SQLAlchemy, INFORMATION_SCHEMA
ClickHouse, MongoDB	system tables, sampled documents
Metabase	its REST metadata
Anything else	`fetch` through a read-only command

Result: a folder with `graph.json`, `keygraph.db` (SQLite with full-text search), `keygraph.html` (the explorer), `RELATIONS_REPORT.md` and the agent pages in `areas/` and `tables/`.

STEP 2A

Keys and indexes are read straight from the schema

Before any guessing, keygraph records what the database says. That tells you which columns are safe to filter on, and which indexes are wasted.

```
$ keygraph keys shop.orders --dir examples/shop/out
shop.orders
  primary key: id
  unique keys: none
  indexes (column order matters; filter on the leading column first):
    PRIMARY [primary]: id
    ix_orders_customer_id: customer_id
    ix_orders_customer_placed: customer_id, placed_at
    ix_orders_placed: placed_at
  redundant: ix_orders_customer_id is covered by
ix_orders_customer_placed (prefix of a longer index)
```

How to read it

- **Column order matters.** An index on (customer_id, placed_at) helps a filter on customer_id, or on both. It does not help a filter on placed_at alone. That is why the list shows each index in column order.
- **Efficient filters** are the leading columns of the primary key, unique keys and indexes. Filter on these first when a table has millions of rows.
- **Redundant indexes** are flagged. Here `ix_orders_customer_id` is a prefix of `ix_orders_customer_placed`, so it only costs write speed.
- **Table facts** such as engine, row estimate and comment come from `--tables`. The row estimate (5.2M) is what makes the explorer warn before you scan a big table.

> The same facts, plus column types, defaults, enum members and comments, are in `keygraph describe` and on each table page of the explorer.

STEP 2B

Missing foreign keys are inferred, with a score and the reasons

Many databases declare few foreign keys. keygraph matches columns such as order_id to a table called orders, scores the match, and shows every reason it used.

```
$ keygraph explain billing.payments.order_id --dir examples/shop/out
billing.payments.order_id  (bigint unsigned NOT NULL)
  keys: index ix_payments_order (position 1 of 1)
  references:
    shop.orders(id)  [inferred 0.95, many-to-one]
        - column name 'order_id' matches table name
        - compatible types (int ~ int)
        - source column is the leading column of index
'ix_payments_order'
        - target is the primary key of shop.orders: id
        - cross-schema match
        - business relation: order is paid by payment (source:
examples/shop/context.yaml)
```

This relation is not declared in the demo. It crosses two schemas, which lowers the score a little, and the index on the column raises it.

What changes the score

RAISES IT	LOWERS IT
Declared foreign key (always 1.0)	Type mismatch
Name matches a table	Match across schemas
Target is a primary key	More than one possible target
Index on the column	Values barely overlap (after `probe`)
A business relation in the context file	No key on the target

Read the number like this. 0.8 and above: use it. 0.6 to 0.8: read the evidence first. Below 0.6: hidden until you switch on "Show weak relations" (floor 0.4), and check it with a small SELECT.

Role columns such as `created_by_id` are recognised as pointers to a staff table. Polymorphic pairs such as `target_type` + `target_id` have no single target and are marked as such.



STEP 3

Hubs and areas turn 35 tables into 4 pictures

A table that half the database points to would make every drawing unreadable. keygraph shows such hubs as labels, groups the rest into business areas, and keeps copied schemas out of the way.

```
$ keygraph areas --dir examples/shop/out
a_11f94d26  Catalog  (8 tables, curated): shop.products, shop.sellers,
shop.product_variants
a_a66985fc  Ordering  (8 tables, curated): shop.orders, shop.customers,
shop.order_items
a_c45fed59  Payments and vouchers  (6 tables, curated): billing.payments,
billing.vouchers, billing.invoices
a_a1dba317  Fulfilment  (5 tables, curated): shop.shipments, shop.inventory,
shop.returns
a_595a9d55  Unconnected (shop)  (3 tables): shop.audit_log, shop.comments,
shop.users
```

```
$ keygraph area Ordering --dir examples/shop/out
Ordering [a_a66985fc] (curated): 8 tables

tables:
  shop.orders - one row per order
  shop.customers
  shop.order_items - one row per variant on an order
  shop.cart_items
  shop.carts
  shop.customer_addresses
  shop.order_status_history
  shop.orders_shadow - one row per order at the time of a one-off migration

internal relations (8):
  shop.cart_items(cart_id) -> shop.carts  conf 1.00
  shop.carts(customer_id) -> shop.customers  conf 0.95
  shop.customer_addresses(customer_id) -> shop.customers  conf 1.00
  shop.order_items(order_id) -> shop.orders  conf 1.00
  shop.order_status_history(order_id) -> shop.orders  conf 0.95
  shop.orders(address_id) -> shop.customer_addresses  conf 0.58
  shop.orders(customer_id) -> shop.customers  conf 1.00
  shop.orders_shadow(customer_id) -> shop.customers  conf 0.95

neighbouring areas:
  Payments and vouchers [a_c45fed59]: 5 relations
  Catalog [a_11f94d26]: 4 relations
```

Hubs

A hub is a table in the top 1% by incoming relations, or one where at least 60% of the incoming relations come from role columns such as `created_by_id`. In the demo, `shop.users` receives 13 such relations. It appears as a small "users" label on each box instead of 13 lines.

Areas

Tables that join each other form an area. The grouping is deterministic: the same dumps always give the same areas. A short curation in `context.yaml` renames or moves tables, so "Ordering" and "Fulfilment" read the way your team talks. Tables with no relations once hubs are removed land in "Unconnected".

Mirrors

`reporting` holds copies of four `shop` tables. keygraph detects that, marks the schema as a mirror of `shop`, and leaves it out of areas, hub counts and join paths so nothing is counted twice. `--include-mirrors` brings it back.

```
areas:
  - name: Fulfilment
    pin: [shop.shipments, shop.shipment_items,
          shop.warehouses, shop.inventory, shop.returns]
```

STEP 4A

Business context says what the tables mean

A schema knows column names. It does not know that status 3 means shipped or that orders_shadow is scratch space. You write that down once, in a YAML file, and keygraph keeps it next to the schema.

```
concepts:
  - name: customer
    definition: A person with an account who can place orders.
    synonyms: [buyer, shopper, account holder]
    links: [{table: shop.customers, role: primary-record}]
columns:
  - table: shop.orders
    column: status
    meaning: Order lifecycle state
    codes: {"1": placed, "2": paid, "3": shipped, "4": cancelled}
metrics:
  - name: net revenue
    expression: SUM(order_items.qty * order_items.unit_price -
order_items.discount)
    tables: [shop.order_items, shop.orders]
tables:
  - table: shop.order_items
    grain: one row per variant on an order
    pitfalls: Joining this table to shipments repeats each line.
```

Also supported: concept relations, `pii: true` flags, markdown notes per table (`context import-notes`) and `context add` on the command line. Unknown tables only warn.

```
$ keygraph search revenue --dir examples/shop/out
Search: revenue
  metrics: average order value, net revenue
1. shop.orders  (score 15.78, verified)
     grain/summary: one row per order
     pk: id   filter on: id, customer_id, placed_at
     why: metric 'net revenue'; metric 'average order value'; table notes
match
     pitfalls: Status 1 rows are baskets that are not paid yet, so leave
them out of revenue.
2. shop.order_items  (score 7.51, verified)
     grain/summary: one row per variant on an order
     pk: id   filter on: id, seller_id, order_id, variant_id
     why: metric 'net revenue'
```

Search ranks tables by names, synonyms, concepts, metrics, notes, column codes and profiled values. Each hit says why it matched, what one row is, which columns to filter on and what to watch out for.

> Every context row records its source file and whether someone verified it, and it can be exported again with `keygraph context export`.

STEP 4B

Value profiles show what is really in the columns

The context file says what should be there. A profile samples the data and shows what is. It also compares the two, and it never stores personal values.

```
keygraph profile --dir out --source sqlite \
  --url sqlite:///shop.db --bare-tables \
  --tables shop.customers,shop.orders,shop.order_items
```

```
$ keygraph profile ... (output)
{
 "requested": 3,
 "profiled": 3,
 "columns": 27,
 "skipped": {},
 "failed": {},
 "findings": [
  "shop.orders.status: values in data but undocumented: 9"
 ],
 "seconds": 0.0
}
```

One bounded SELECT per table, never a full scan. Each profile records when it was taken, how many rows and by which method, so a reader knows how fresh and approximate it is.

Personal data stays out

```
$ keygraph describe shop.customers  (3 of 8 columns)
    email varchar(190) NOT NULL - Login and contact address [PII]  [60
rows sampled (recent n=10000, 2026-11-04); null 0%; 60 distinct; PII,
values withheld; looks like email]
    first_name varchar(80) NOT NULL  [60 rows sampled (recent n=10000,
2026-11-04); null 0%; 8 distinct; PII, values withheld]
    status tinyint NOT NULL - Account state codes: 1=active, 2=blocked
[60 rows sampled (recent n=10000, 2026-11-04); null 0%; 2 distinct; range
1..2, median 1; top: 1 (47), 2 (13)]
```

PII columns are recognised by name (email, phone, name, address, token …), by value pattern, or by a `pii` flag. For them keygraph keeps counts, empty rate, length and the pattern name. It never stores a top value, a minimum or a maximum.

STEP 5

Code and past queries link back to the same tables

keygraph can read your repositories as text and log the queries people and agents ran. Both are linked to the tables and columns of the graph, so a column page answers where it is used and how it was queried.

Where a column lives in code

```
$ keygraph code where shop.orders.status
shop.orders.status: 5 code refs (declare_column 1, sql_write 1, sql_read
1, use 2)
  shop-app:shop/models.py:20  declare_column shop.orders.status  1.00
sqlalchemy-column  status
  shop-app:shop/orders.py:17  sql_write      shop.orders.status  0.95
sql:sqlglot
  shop-app:shop/orders.py:8  sql_read       shop.orders.status  0.95
sql:sqlglot
  shop-app:shop/orders.py:12  use            shop.orders.status  0.40
use:symbol  status
  shop-app:shop/orders.py:13  use            shop.orders.status  0.40
use:symbol  status
```

ORM declarations come first, then SQL that writes and reads the column, then uses of its property. Each line names the rule that found it and a confidence. Code is never imported or run.

How the table was queried before

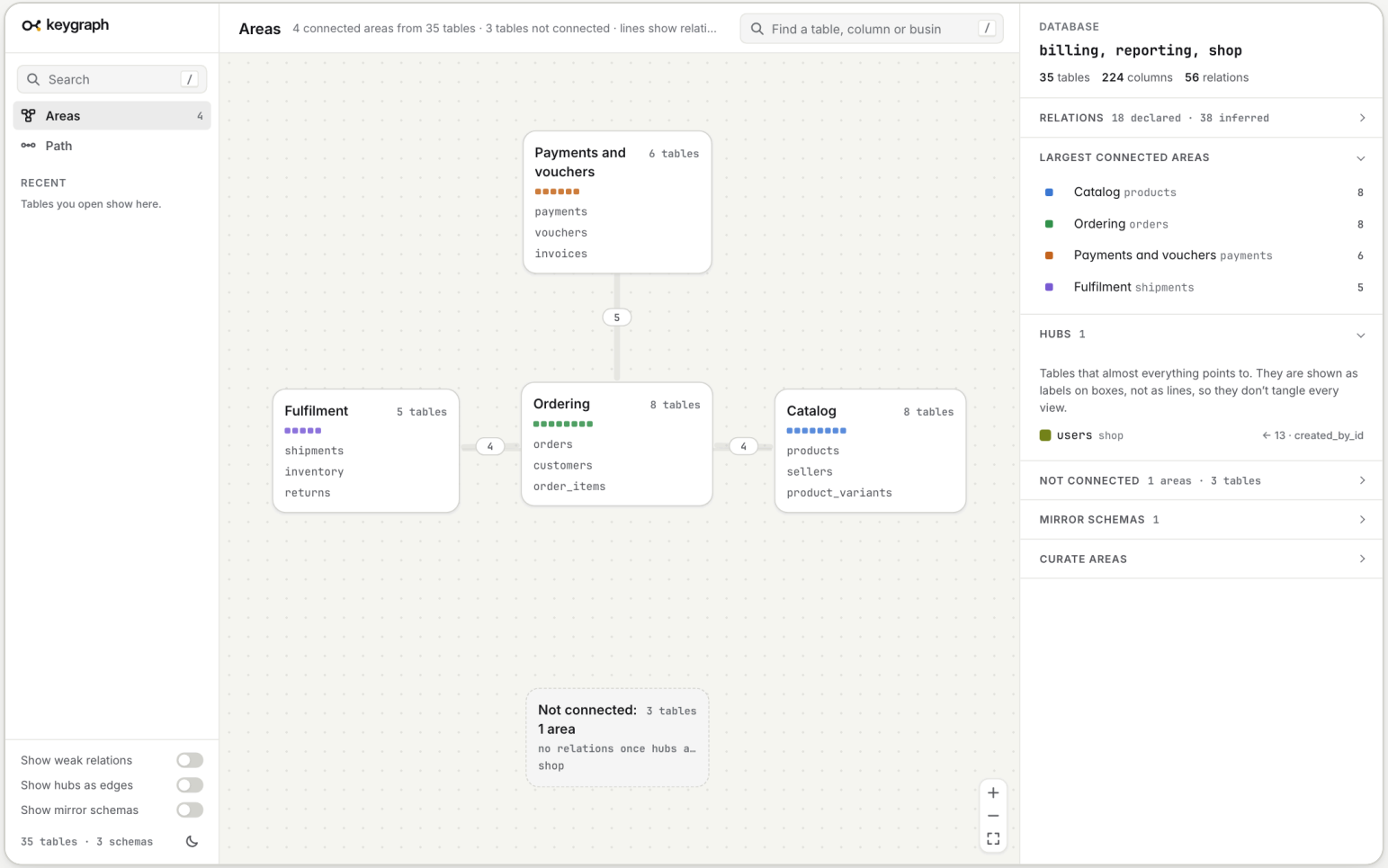
```
$ keygraph queries top --table shop.orders
top queries by runs touching shop.orders
0887c908651c57e4  3 runs, 2 people, median 47 ms, p95 51 ms
  why: Paid orders this month
  sql: select o.id, o.total from shop.orders o where o.status = ? and
o.placed_at >= ?
381600506fcebf80  1 runs, 1 people, median 380 ms, p95 380 ms
  why: Top customers by number of orders
  sql: select c.id, count (*) as orders from shop.customers c join
shop.orders o on o.customer_id = c.id group by c.id order by orders desc
limit ?
```

Queries that differ only in literal values share one fingerprint. Literals are masked before storing, and each query keeps the business question it answered, its run count and its median and p95 time.

EXPLORER 1 OF 6 · AREAS

The Areas view is the map of the whole database

Start here when you do not know where something lives. Each box is a business area; the number on a line is how many relations join two areas.

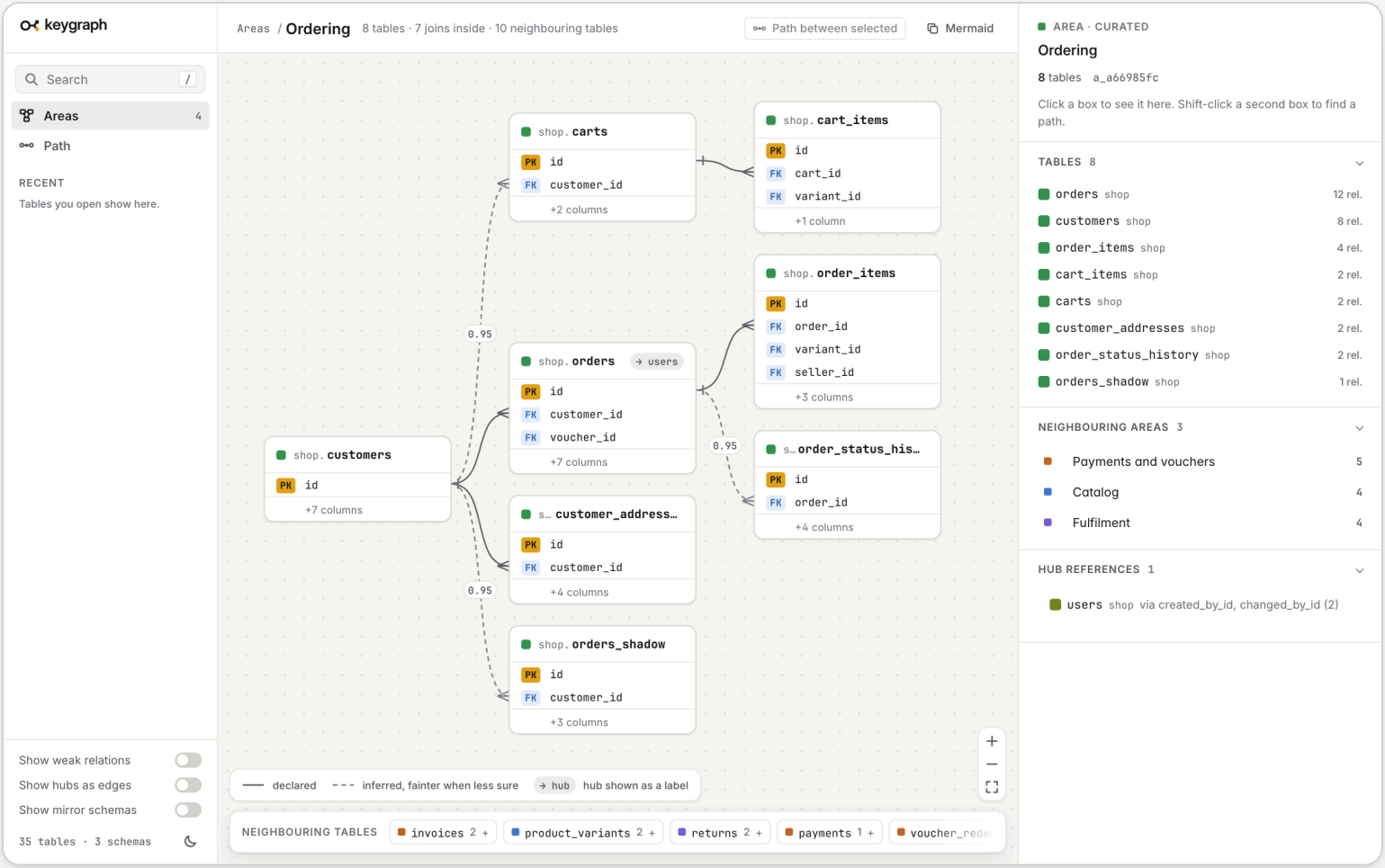


What to look for

- Each box shows the area name, its table count, one square per table and its three busiest tables.
- Line thickness follows the number of relations between two areas. Hubs never draw lines here.
- The side panel counts relations by kind, lists the hubs and the mirror schema, and opens the area curation help.
- Click a box to open that area.

The Area view draws one area as a table diagram

Boxes are tables listing their key columns. Lines attach to the exact column that joins, so you can see which column does the work.

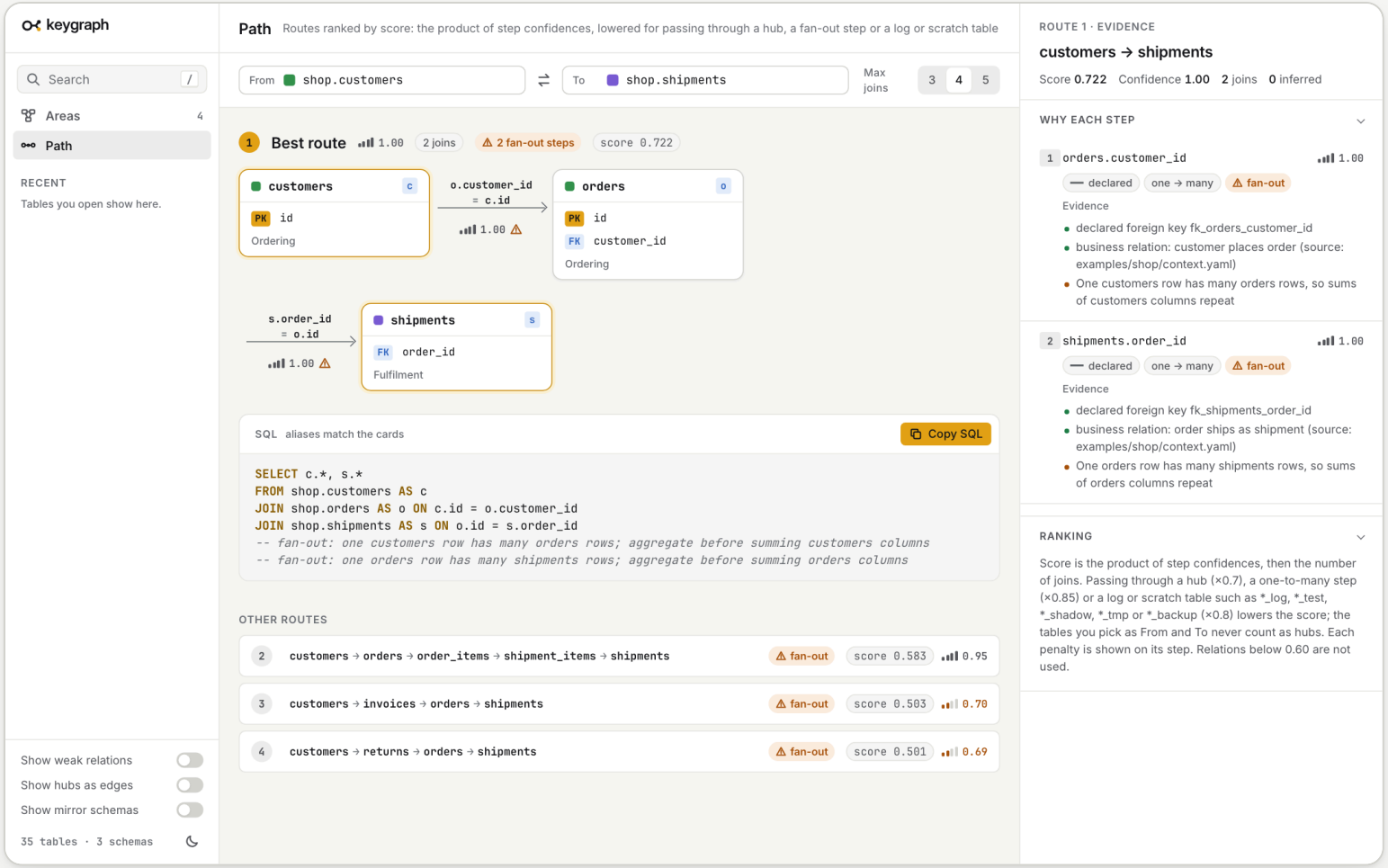


What to look for

- Solid lines are declared foreign keys. Dashed lines are inferred, with the confidence on the line.
- The pill on `orders` reads 'users': the hub reference, shown as a label and not as a line.
- '+7 columns' says how much is hidden. Open the table to see all of it.
- Neighbouring tables from other areas wait in the strip at the bottom. An area never draws more than about 40 boxes, and over 25 tables it first shows the 25 most connected.

The Path view finds the join between two tables

Pick two tables and get ranked routes, a copyable JOIN, and a warning at every step that can multiply rows.



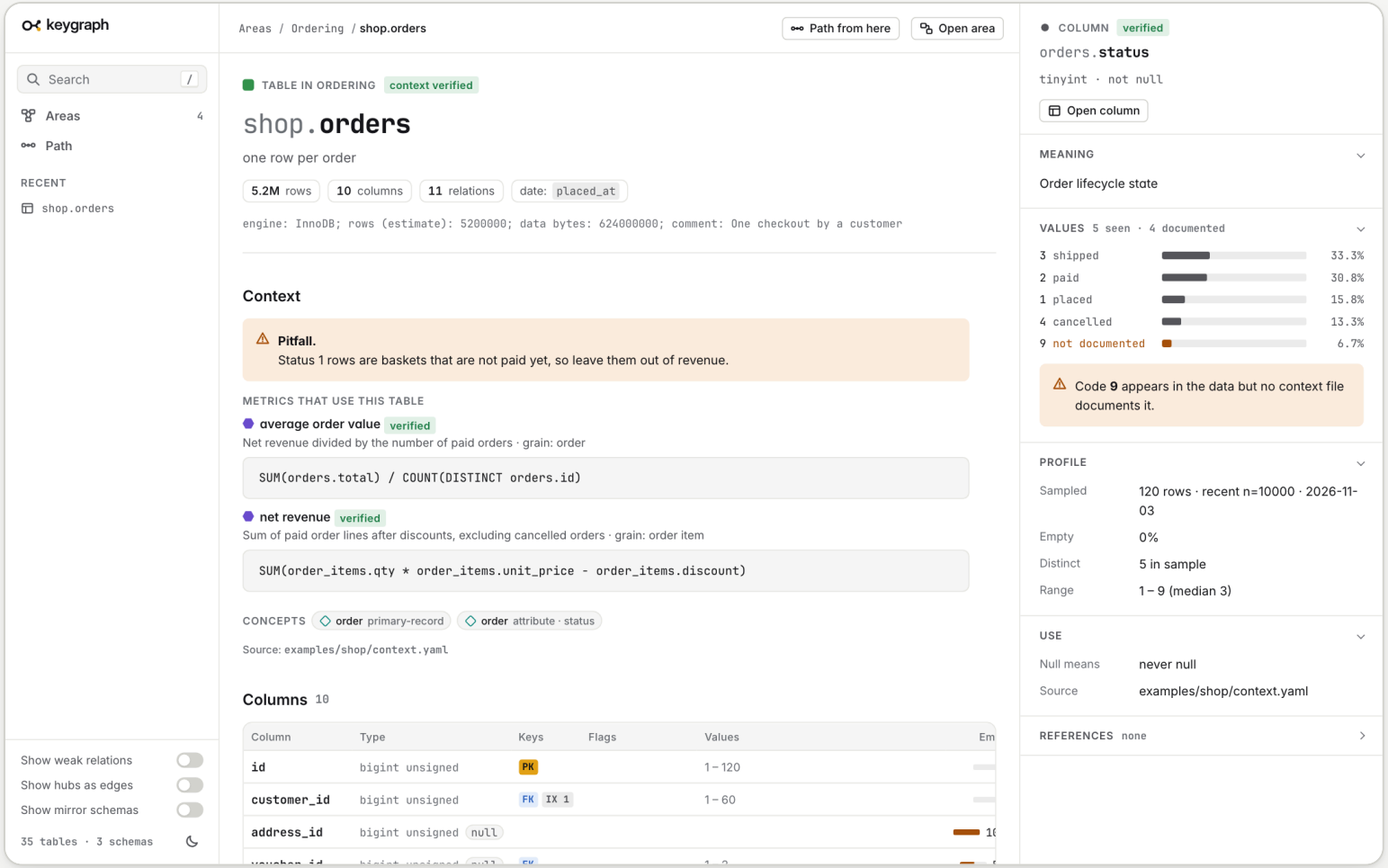
What to look for

- The best route is customers, orders, shipments: two declared joins.
- Both steps carry a fan-out warning. One customer has many orders, so summing a customer column over this join counts it many times. The SQL comment says to aggregate first.
- Routes through a hub, a one-to-many step or a scratch table (`*_shadow`, `*_log`) score lower. The side panel shows each step's evidence.

EXPLORER 4 OF 6 · TABLE

The Table view explains one table as a page

This is the page to read before you query a table: what a row is, what to avoid, which metrics use it and how its columns look.



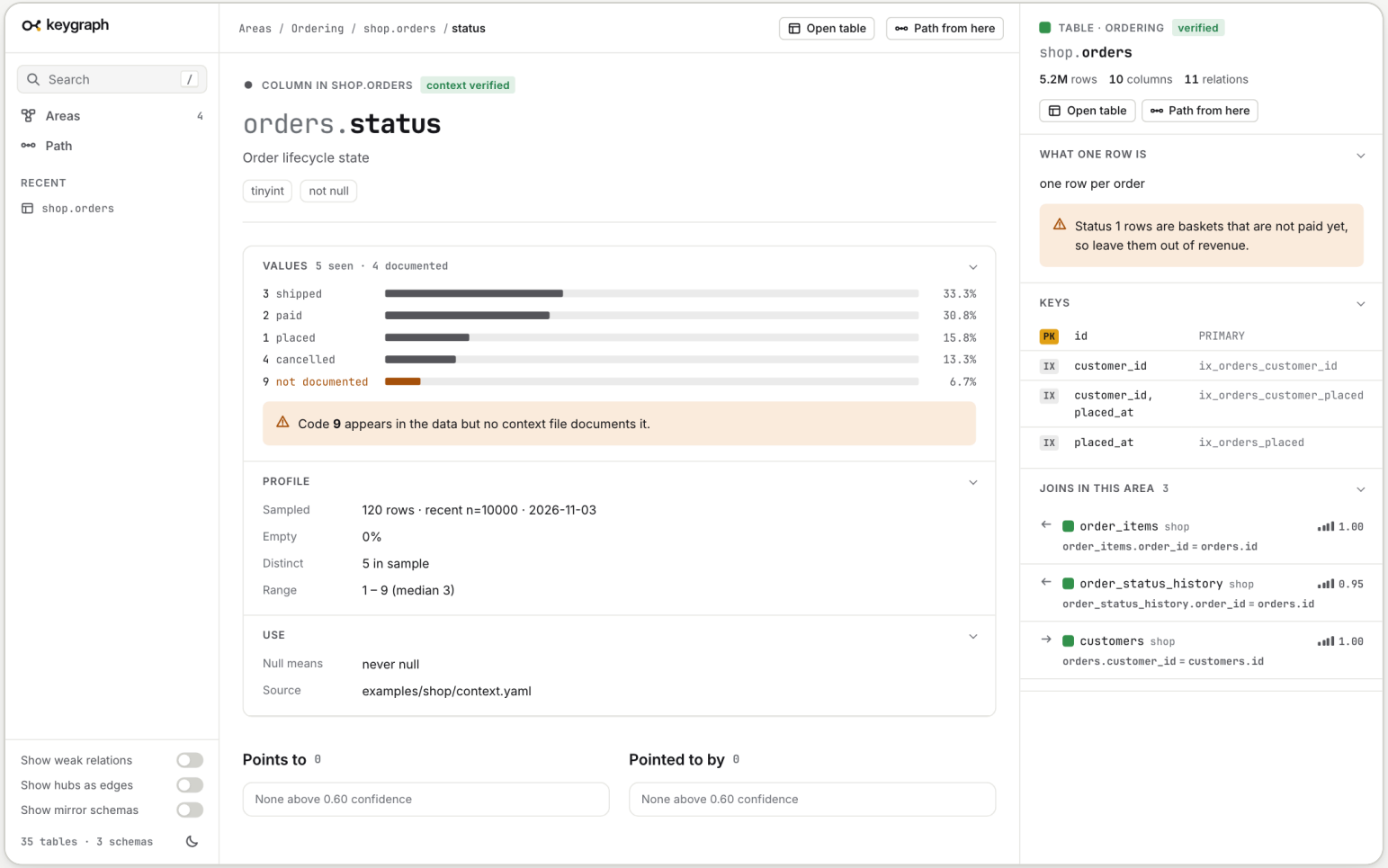
What to look for

- The header gives the row grain, row estimate, column and relation counts and the date column.
- The orange box is the pitfall from the context file. The purple items are metrics that use this table, with their expressions.
- The column grid shows keys, flags, value ranges and how often a column is empty.
- Relations below the grid are grouped by column and area; groups over 8 items collapse.

EXPLORER 5 OF 6 · COLUMN

The Column view shows what a column holds and how it is used

For status columns this is where the documented codes meet the real data.

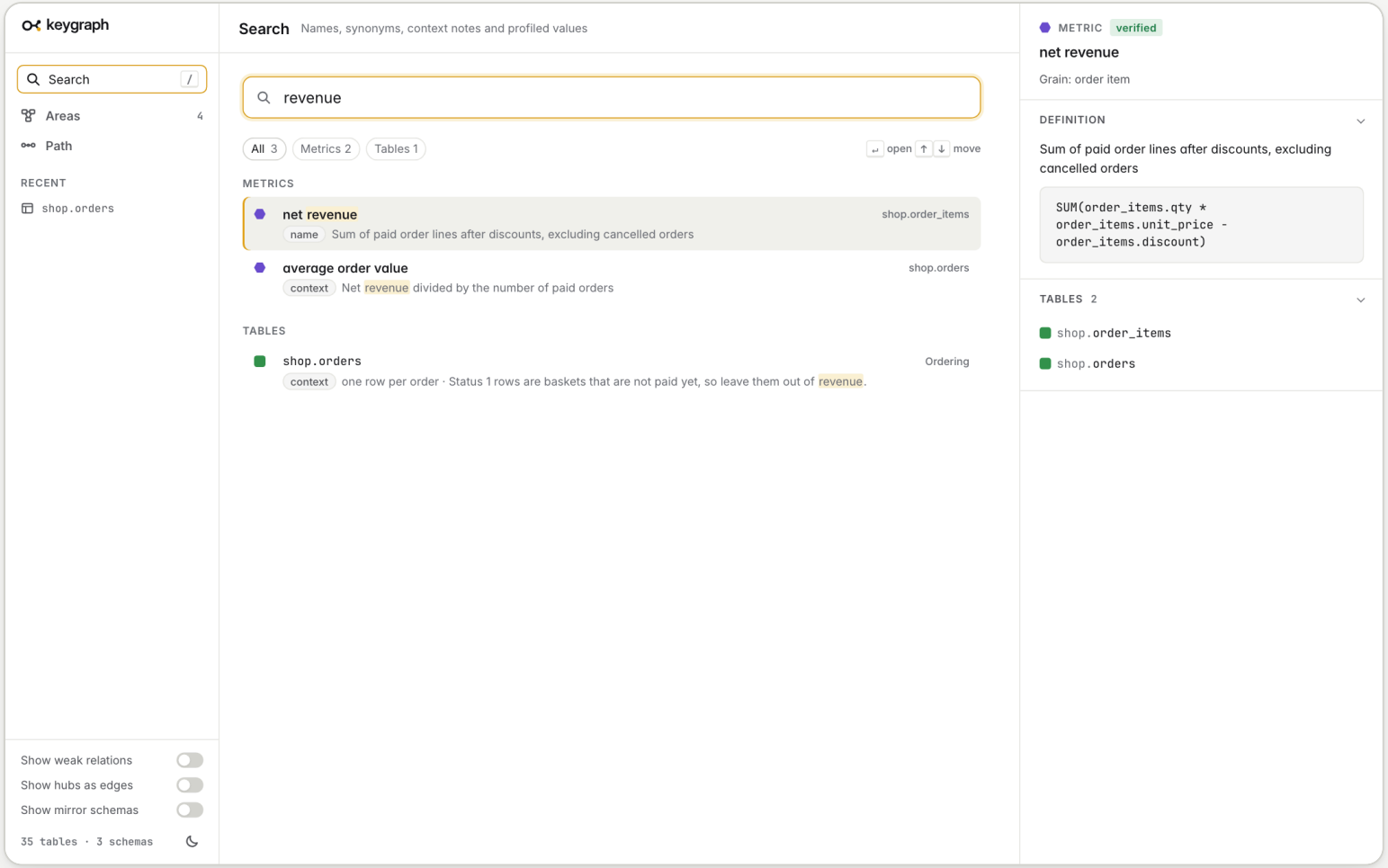


What to look for

- The bars compare the documented codes with the sampled values. Code 9 appears in 6.7% of rows and the context file does not know it, so it is flagged.
- The profile block says when and how the sample was taken. 'Null means' comes from the context file.
- The side panel lists the indexes that cover the column and the joins it takes part in.

Search finds tables, columns, concepts and values by meaning

Type a business word. Results are grouped by type and each says why it matched.



What to look for

- Metrics, concepts, tables, columns and profiled values come from one index, so a status label such as 'in transit' finds the column that stores it.
- A reason chip shows the match: name, synonym, context, enum label or profiled value.
- Press Enter to open the top result in the view that fits it. The side panel previews the highlighted one.

FOR AGENTS

An agent uses the same graph through a short command sequence

Every command prints plain text or JSON (`--json`), and the build also writes markdown and Mermaid pages that an agent can read like any other file.

The sequence

NEED	COMMAND
Where does this live?	`keygraph search "net revenue"`
What is around it?	`keygraph areas` , `keygraph area Ordering`
What is this table?	`keygraph describe shop.orders`
How do I join A to B?	`keygraph path A B`
Why this relation?	`keygraph explain T.col`
Which filter is fast?	`keygraph keys T`

```
$ keygraph path shop.customers shop.shipments --dir examples/shop/out -k
1
Path 1: shop.customers -> shop.orders -> shop.shipments   (score 0.722,
confidence 1.0, 2 hops)
  shop.customers(id) -> shop.orders(customer_id)  [declared 1.00, one-to-
many]  [!] fan-out
      - declared foreign key fk_orders_customer_id
  shop.orders(id) -> shop.shipments(order_id)  [declared 1.00, one-to-
many]  [!] fan-out
      - declared foreign key fk_shipments_order_id

  FROM shop.customers AS c
  JOIN shop.orders AS o ON c.id = o.customer_id
  JOIN shop.shipments AS s ON o.id = s.order_id
```

Files an agent can read

```
areas/index.md         every area with its top tables
areas/ordering.md      tables, joins, Mermaid ER diagram
tables/shop.orders.md  grain, keys, columns,
                       points to / pointed to by
RELATIONS_REPORT.md    all relations with evidence
graph.json             everything, machine readable
```

```
erDiagram
    orders {
        bigint_unsigned id PK
        bigint_unsigned customer_id FK
    }
    %% orders hub ref to shop.users via created_by_id
```

The `keygraph` skill in `skills/keygraph/SKILL.md` tells an agent when to use each command and how to read confidence: use 0.8 and above freely, check 0.6 to 0.8 against the evidence, and verify anything lower, ambiguous or marked fan-out with a small SELECT.

> Corrections go back in: `context add` and an overrides file (`confirm` , `reject` , `add`) make the next build better.